MODULE 01 | TERMINAL SHELL

Linux Basics

Your first steps in the terminal: find where you are, look around, create things, move between folders and ask for help.

Mission	Topic	Commands
1	Explore Your Terminal	pwd, ls, echo, redirection (>, >>)
2	Build Your Workspace	mkdir, touch, rmdir
3	Move Around	cd, absolute vs relative paths, ~ . ..
4	Ask for Help	man, --help, whatis, which
5	Work Faster	Tab, history, clear, whoami, Ctrl shortcuts
6	Boss: Putting It Together	Everything above

How to use this guide: read each mission, type every example yourself in the Tux Tutor terminal, then attempt the practice tasks before checking the answers at the end. Typing is how muscle memory forms.

Before You Start

What is the terminal?

The terminal is a text window where you give instructions to your computer by typing commands. The program that reads and runs those commands is called the **shell** (usually Bash). Instead of clicking through folders, you type short commands such as `ls` to list files.

Why learn it?

- Servers, cloud machines and containers are managed almost entirely through the terminal.
- Commands can be repeated, combined and automated, which clicking cannot do.
- It is a core skill for developers, DevOps, data engineers and sysadmins.

Anatomy of a command

```
command  -options  arguments

ls       -l       /home
^        ^        ^
|        |        what to act on
|        changes how it behaves
what to do
```

- **Command**: the program to run.
- **Option (flag)**: modifies behaviour. Short form `-l`, long form `--long`. Short flags combine: `-la` = `-l -a`.
- **Argument**: the file or folder the command works on.

> **Remember**
> Linux is **case-sensitive**. `Notes.txt`, `notes.txt` and `NOTES.txt` are three different files.

The prompt

```
haani@tux:~$

haani   -> your username
tux     -> the machine name
~       -> your current location (~ means home folder)
$       -> you are a normal user (# means root/admin)
```

MISSION 1 | +50 XP

Explore Your Terminal: pwd, ls and Redirection

Goal: Know where you are, see what is around you, and save output to a file.

pwd - Print Working Directory

Shows the full path of the folder you are currently in. Run it whenever you feel lost.

```
$ pwd
/home/haani
```

ls - List

Lists the contents of a directory. Options make it far more useful:

Command	What it does
`ls`	Names of files and folders
`ls -l`	Long format: permissions, owner, size, date
`ls -a`	Include hidden files (names starting with a dot)
`ls -la`	Long format plus hidden files
`ls -lh`	Human-readable sizes (KB, MB)
`ls -R`	Recursive: list subfolders too
`ls -t`	Sort by modification time, newest first
`ls /etc`	List a different folder without moving there

Reading ls -l output

```
-rw-r--r-- 1 haani haani 1420 Nov 14 10:30 notes.txt
drwxr-xr-x 2 haani haani 4096 Nov 14 10:28 projects

first char : - = file, d = directory, l = link
next 9     : permissions (covered in Module 04)
1420       : size in bytes
Nov 14...  : last modified
```

echo - Print Text

Displays text. On its own it looks trivial, but it becomes powerful with redirection.

```
$ echo "Hello, Tux!"
Hello, Tux!
```

Redirection: sending output to a file

Normally output goes to the screen. Redirection sends it somewhere else.

Operator	Meaning
>	Write output to a file. **Overwrites** existing content.
>>	**Append** output to the end of a file.

```
$ ls -la > listing.txt          # save listing to a file
$ echo "Day 1" > log.txt         # creates/overwrites log.txt
$ echo "Day 2" >> log.txt        # adds a second line
$ cat log.txt                    # show file contents
Day 1
Day 2
```

Common mistake
Using > when you meant >> silently wipes the file. Linux will not ask for confirmation.

Practice

- Print your current directory and save it to `where.txt`.
- List all files (including hidden) in long format and append that to `where.txt`.
- Which hidden files exist in your home folder?

MISSION 2 | +50 XP

Build Your Workspace: mkdir, touch and rmdir

Goal: Create folders and files, and organise a project structure.

mkdir - Make Directory

```
$ mkdir projects                # one folder
$ mkdir logs backups            # several at once
$ mkdir -p web/css/fonts        # create nested path in one go
$ mkdir -v test                 # -v prints what it did
```

The `-p` (parents) flag creates every missing folder along the path. Without it, `mkdir web/css/fonts` fails if `web` does not exist yet.

touch - Create Empty Files

`touch` creates an empty file if it does not exist. If it does exist, it only updates the modified timestamp, so existing content is safe.

```
$ touch notes.txt
$ touch a.txt b.txt c.txt       # several files
$ touch report_{1..3}.txt       # brace expansion: report_1, _2, _3
$ ls
a.txt b.txt c.txt notes.txt report_1.txt report_2.txt report_3.txt
```

rmdir - Remove Empty Directory

Deletes a folder only if it is empty, which makes it a safe way to clean up. (Deleting folders with content uses `rm -r`, covered in Module 02.)

```
$ rmdir test
$ rmdir web
rmdir: failed to remove 'web': Directory not empty
```

Naming rules and good habits

- Avoid spaces. Use `my_notes.txt` or `my-notes.txt`. If you must use spaces, quote the name: `"my notes.txt"`.
- Prefer lowercase letters, digits, `_`, `-` and `.`.
- A leading dot hides a file: `.config`.
- Linux does not rely on extensions, but they help humans (`.txt`, `.sh`, `.py`).

Example project skeleton

```
$ mkdir -p myapp/{src,docs,tests}
$ touch myapp/src/main.py myapp/docs/README.md
$ ls -R myapp
myapp:
docs  src  tests

myapp/docs:
README.md

myapp/src:
main.py

myapp/tests:
```

Practice

- Create `workspace/` with three subfolders `notes`, `code`, `backup` using one command.
- Create five files `day1.txt` to `day5.txt` in `notes` using brace expansion.
- Try to `rmdir workspace`. Why does it fail?

MISSION 3 | +50 XP

Move Around: cd and Paths

Goal: Navigate the filesystem confidently using absolute and relative paths.

The Linux directory tree

Everything starts at the root directory `/`. There are no drive letters like C: or D:.

Path	Purpose
`/`	Root of the whole filesystem
`/home`	Users' personal folders
`/etc`	System configuration files
`/var`	Variable data such as logs
`/tmp`	Temporary files
`/usr`	Installed programs and libraries
`/bin`	Essential commands (ls, cp, ...)
`/root`	Home of the root (admin) user

cd - Change Directory

```
$ cd projects          # go into projects
$ cd ..                # up one level (parent)
$ cd ../..             # up two levels
$ cd ~                 # home folder
$ cd                   # also home
$ cd -                 # back to previous folder
$ cd /var/log          # absolute path
```

Special path symbols

Symbol	Means
`.`	Current directory
`..`	Parent directory
`~`	Your home directory
`/`	Root (when at start of a path)
`-`	Previous directory (with cd)

Absolute vs relative paths

An **absolute** path starts with `/` and works from anywhere. A **relative** path starts from where you are now.

```
# You are in /home/haani
$ cd /home/haani/projects/web    # absolute
$ cd projects/web                # relative (same result)

# Now in /home/haani/projects/web
$ cd ../../docs                  # relative: up 2, then into docs
$ pwd
/home/haani/docs
```

Tip
Not sure where you ended up? Run `pwd`. Want to go home no matter what? Run `cd` with nothing after it.

Common errors

```
$ cd Projects
bash: cd: Projects: No such file or directory   # wrong case!

$ cd my notes
bash: cd: too many arguments                    # space splits it
$ cd "my notes"                                 # fix: quote it
```

Practice

- From anywhere, go to `/var/log` in one command, then back to where you were.
- From `/usr/share`, reach `/etc` using a relative path.
- Use `cd -` to bounce between two folders.

MISSION 4 | +50 XP

Ask for Help: man, --help and friends

Goal: Learn to find answers yourself, the most valuable Linux skill.

man - the manual

Almost every command has a manual page. Open one with `man`:

```
$ man ls
```

Key inside man	Action
Space / f	Next page
b	Previous page
Arrow keys	Scroll line by line
/word	Search for 'word' (n = next match, N = previous)
q	Quit

Sections of a man page

- **NAME**: one-line description.
- **SYNOPSIS**: how to write the command. `[ ]` means optional, `...` means repeatable.
- **DESCRIPTION**: what each option does.
- **EXAMPLES / SEE ALSO**: related commands.

Quicker alternatives

```
$ ls --help            # short built-in summary
$ whatis ls             # one-line description
ls (1) - list directory contents
$ man -k copy           # search manuals by keyword
$ which ls              # where is this command located?
/usr/bin/ls
$ type cd               # is it a program or shell built-in?
cd is a shell builtin
```

Tip: `cd` and some others are shell built-ins, so use `help cd` instead of `man cd`.

> **Pro habit**
> When you meet an unknown command, run `whatis` first, then `--help`, then `man` only if you need depth.

Practice

- Use `man ls` to find which option sorts by file size.
- Use `man mkdir` to find what `-m` does.
- Find the location of the `bash` program with `which`.

MISSION 5 | +50 XP

Work Faster: Shortcuts and Helper Commands

Goal: Type less, repeat less, and keep your screen tidy.

Tab completion

Type the first letters and press **Tab** to auto-complete a name. Press Tab twice to see all matches. This prevents typos and saves enormous time.

```
$ cd Doc<Tab>        ->   cd Documents/
```

history

```
$ history            # numbered list of past commands
$ !!                 # repeat the last command
$ !25                # run command number 25
$ history | tail -5   # last five commands
```

Press the **Up arrow** to scroll through earlier commands, or **Ctrl+R** to search them by typing part of the command.

Other helpers

Command	Use
`clear`	Clear the screen (or press Ctrl+L)
`whoami`	Show the current username
`hostname`	Show the machine name
`date`	Current date and time
`uname -a`	Kernel and system information
`cat file`	Print a file's contents
`exit`	Close the shell session

Keyboard shortcuts

Shortcut	Action
Ctrl+C	Stop the running command
Ctrl+L	Clear the screen
Ctrl+A / Ctrl+E	Jump to start / end of the line
Ctrl+U	Delete from cursor to start of line
Ctrl+W	Delete the previous word
Ctrl+R	Search command history
Ctrl+D	Exit shell (end of input)

Wildcards (a first taste)

```
$ ls *.txt          # every file ending in .txt
$ ls report_?.txt   # ? matches exactly one character
$ ls day[1-3].txt   # day1, day2 or day3
```

Practice

- Type a long command wrong on purpose, then fix it using Ctrl+A and arrow keys.
- Re-run your previous command with `!!`.
- List only `.txt` files in a folder using a wildcard.

BOSS BATTLE | +150 XP

Boss: The Lost Workspace

Goal: combine everything from Module 01 without hints.

Scenario

A new project folder is needed, and your teammate left instructions. Complete each task using only what you have learned.

Tasks

1. Print your current directory and save it to `~/mission.txt`.
2. Create `~/lab/{docs,src,logs}` with a single command.
3. Inside `logs`, create `app.log` and `error.log`.
4. Move into `src` using a relative path, then confirm with `pwd`.
5. From `src`, jump to `docs` in one command using `..`.
6. Write the line `Lab ready` into `docs/README.md`, then append the current date after it.
7. Use `ls -R ~/lab` to verify the whole structure.
8. Find, using the manual, the `ls` option that reverses sort order, and run it.

Sample solution

```
pwd > ~/mission.txt
mkdir -p ~/lab/{docs,src,logs}
touch ~/lab/logs/app.log ~/lab/logs/error.log
cd ~/lab/src
pwd
cd ../docs
echo "Lab ready" > README.md
date >> README.md
ls -R ~/lab
ls -r ~/lab          # -r reverses the sort order
```

Check Your Understanding

Q1. Which command shows your current location?

Q2. What is the difference between > and >>?

Q3. Which flag makes `mkdir` create nested folders?

Q4. What does `touch` do to an existing file?

Q5. What does `cd ../..` do?

Q6. What does `cd -` do?

Q7. Name two ways to learn what a command does without leaving the terminal.

Q8. How do you show hidden files?

Q9. What is the hidden-file rule in Linux?

Q10. Why does `cd Projects` fail when the folder is `projects`?

Answers

#	Answer
1	pwd
2	> overwrites the file; >> appends to it.
3	-p
4	Only updates its modification time; content is untouched.
5	Moves up two directory levels.
6	Returns to the previous directory.
7	man, --help, whatis.
8	ls -a (or ls -la).
9	Names beginning with a dot are hidden.
10	Linux is case-sensitive.

Quick Reference Cheat Sheet

Command	Purpose	Example
pwd	Show current directory	pwd
ls	List contents	ls -lah
echo	Print text	echo hi > f.txt
mkdir	Create folder	mkdir -p a/b/c
touch	Create empty file	touch a.txt
rmdir	Remove empty folder	rmdir old
cd	Change directory	cd ../docs

Command	Purpose	Example
man	Open manual	man ls
--help	Quick usage	ls --help
whatis	One-line description	whatis ls
which	Locate a command	which bash
history	Past commands	history | tail
clear	Clear screen	clear
whoami	Current user	whoami

Up next: Module 02
Files & Directories. You will learn cp, mv, rm, find, cat, head and tail to copy, move, delete, search and read files.